



MÓDULO DWEC · DAW/DAM — UNIDAD DIDÁCTICA 1

Arquitecturas Web y Herramientas de Desarrollo

RA1 · JavaScript en el entorno cliente

- Comprender las arquitecturas web y el modelo de ejecución de JavaScript en el navegador
- Dominar DevTools como herramienta principal de depuración
- Configurar un flujo de trabajo profesional con Git desde el primer día



¿Por qué esto importa en el mundo real?

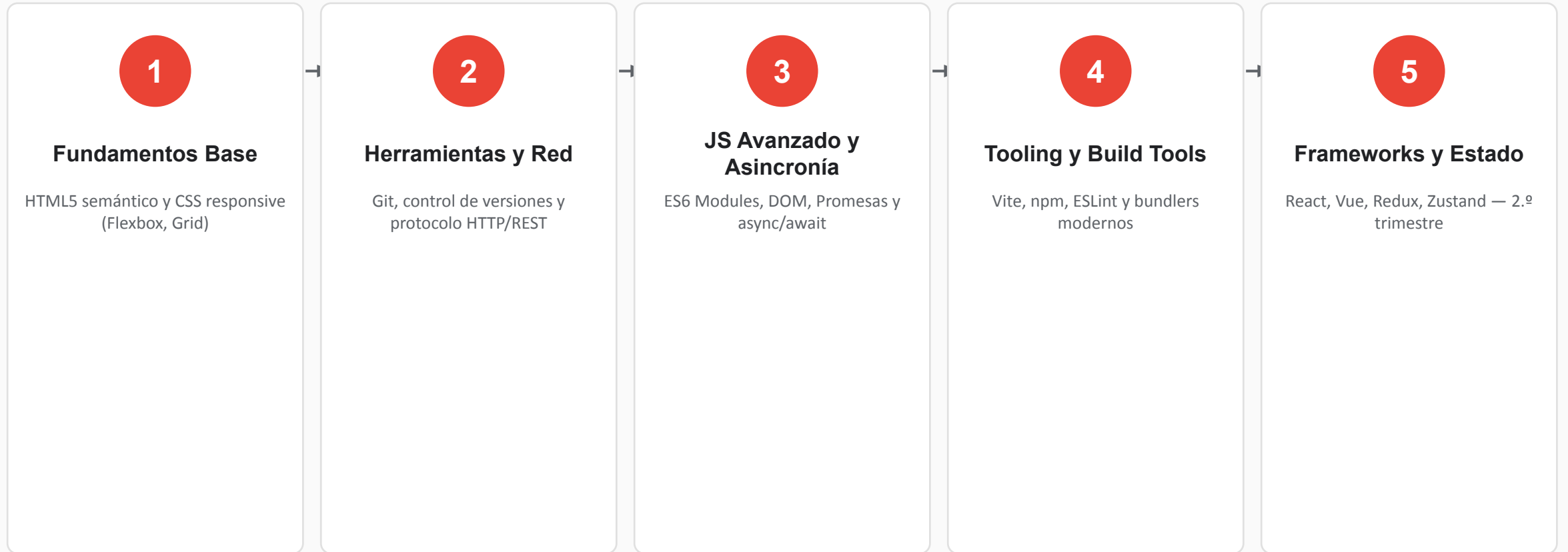
- **JavaScript es universal:** es el único lenguaje que se ejecuta de forma nativa en todos los navegadores
- **Puesto de trabajo real:** un desarrollador frontend usa DevTools y Git literalmente todos los días
- **Bucle de eventos único:** entender cómo funciona realmente evita bugs de concurrencia frecuentes
- **Impacto de negocio:** un código mal depurado en producción genera errores visibles a usuarios reales
- **Base de todo lo que viene después:** React (2º trimestre) es JavaScript — sin esta base, nada de lo posterior tiene sentido



ALERTA COMÚN — El servidor de desarrollo de Django/Flask o el `manage.py runserver` nunca deben usarse en producción — se retomará en IAW, pero aplica igual de raíz aquí con Node.



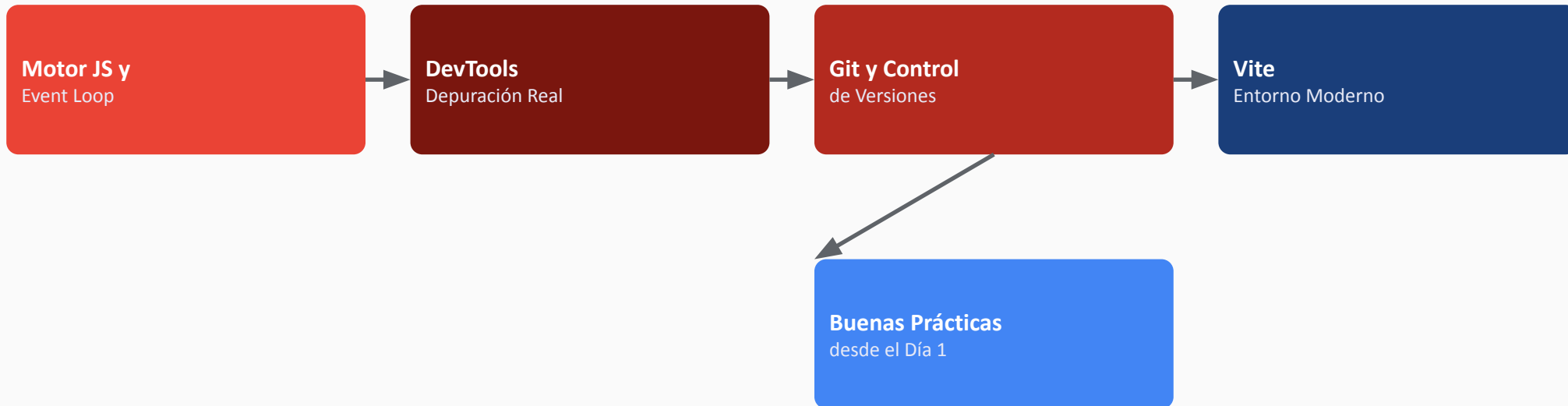
El Roadmap Frontend: Dónde Encajas Hoy



CLAVE — Hoy trabajamos la base del paso 2: sin dominar Git y el modelo cliente-servidor, nada de lo que viene después tiene sentido.



Índice y mapa conceptual de la unidad



CLAVE — Hoja de ruta de hoy: de las arquitecturas web y el modelo cliente-servidor al motor JS, DevTools, Git y herramientas modernas.



Cómo ha Evolucionado la Web



Web Estática (Web 1.0)

El servidor entrega ficheros HTML/CSS intactos, sin lógica de negocio



SSR (Web 2.0)

El servidor genera el HTML en cada petición con PHP, Java o Node.js



SPA (Cliente Pesado)

El navegador ejecuta el JS y consume APIs REST/GraphQL en JSON



Jamstack / Islas

HTML pre-generado que solo hidrata con JS las zonas interactivas

Tendencia clave

La carga de trabajo se ha desplazado progresivamente del servidor al navegador — por eso DWEK (cliente) y DWES (servidor) son módulos distintos.



Frontend vs. Backend: Dos Mundos Distintos

- **Entorno de ejecución:** motor JS del navegador (V8, SpiderMonkey) frente a Node, JVM o PHP en el servidor
- **Confianza y seguridad:** el cliente es un entorno no confiable e inspeccionable; el servidor está protegido tras cortafuegos
- **Recursos:** heterogéneos y limitados (CPU/batería del usuario) frente a clusters homogéneos y escalables
- **Objetivo principal:** UI/UX, validación e interacción frente a reglas de negocio, persistencia y autorización
- **Protocolos:** eventos del DOM y respuestas HTTP/WebSocket frente a peticiones HTTP, gRPC o colas de mensajes



Los Verbos HTTP que Todo Frontend Debe Dominar

- **GET:** solicita un recurso; debe ser idempotente y sin cuerpo en la petición
- **POST:** envía datos al servidor para crear un nuevo recurso
- **PUT:** reemplaza un recurso existente en su totalidad
- **PATCH:** aplica una modificación parcial sobre un recurso
- **DELETE:** elimina un recurso determinado del servidor



Cómo (No) Insertar JavaScript en el HTML

index.html

```
<!-- DESACONSEJADO: antipatrón -->
<button onclick="alert('Hola')">Clic</button>
<script src="app.js" defer></script>

// Preferible: fichero externo con defer
```



NOTA — Evita atributos inline (mezcla HTML/JS). Usa bloques `<script>` internos solo para configuración puntual, y ficheros externos vía `<script src>` para lo demás — permiten caché HTTP y reutilización.



async vs defer: Cómo Usar Cada Uno



Sin Atributos

Bloquea el parseo HTML; se descarga y ejecuta de inmediato



async

Se descarga en paralelo pero interrumpe el HTML al terminar; no respeta el orden



defer

Se descarga en paralelo y se ejecuta al final, respetando el orden de declaración



Recomendación

Usa defer para la inmensa mayoría de scripts de aplicación y librerías

Regla práctica

Si el script necesita el DOM completo o depende de otros scripts, usa defer; async solo para scripts independientes (analítica, anuncios).



Cómo Ejecuta el Navegador tu Código



Motor V8 (Chrome/Node)

Interpreta y compila JavaScript a código máquina en tiempo real



DOM

Representación en árbol del HTML que JavaScript puede leer y modificar



Un solo hilo

JavaScript ejecuta una tarea a la vez, nunca en paralelo dentro del mismo contexto



Event Loop

Gestiona la asincronía sin bloquear la interfaz mientras espera resultados

Analogía útil

Un camarero que toma un pedido, lo envía a cocina y atiende otras mesas mientras espera, en vez de quedarse parado frente a una sola mesa.



Call Stack, Memory Heap y Garbage Collector



Call Stack

Pila LIFO que registra las funciones en ejecución



Memory Heap

Región no estructurada donde se alojan objetos, arrays y funciones



Garbage Collector

Libera memoria automáticamente con algoritmos como Mark-and-Sweep



Stack Overflow

Error al llenarse la pila por una recursión infinita

Dato clave

Si tu función nunca termina de llamarse a sí misma, antes o después el Call Stack se desborda — de ahí el error Stack Overflow.



Microtask vs Macrotask: El Orden de la Cola



Web APIs

Hilos de fondo del navegador para red, temporizadores y DOM



Microtask Queue

Cola de alta prioridad: promesas (.then, async/await)



Macrotask Queue

Cola estándar: setTimeout, clicks y otros eventos



Regla del Event Loop

Vacía TODAS las microtarefas antes de tomar la siguiente macro tarea

Por qué importa

Esto explica por qué una `Promise.resolve().then()` se ejecuta antes que un `setTimeout(fn, 0)`, aunque ambos parezcan 'inmediatos'.



¿Qué es una Promise?



Definición

Un objeto que representa un valor que estará disponible AHORA, MÁS TARDE o NUNCA



Pending

Estado inicial — la operación asíncrona todavía no ha terminado



Fulfilled

Terminó con éxito — dispara las funciones registradas con `.then()`



Rejected

Algo falló — dispara las funciones registradas con `.catch()`

Sintaxis básica

`fetch(url).then(respuesta => ...).catch(error => ...)` — encadenar `.then()` es la base para trabajar con datos que tardan en llegar. Se ve en profundidad en UD6.



Un Vistazo al Orden de Ejecución

orden.js

```
console.log('1');  
setTimeout(() => console.log('2'), 0);  
console.log('3');  
  
// Resultado: 1, 3, 2 -- no 1, 2, 3
```



NOTA — Aunque el timeout sea de 0ms, setTimeout siempre se ejecuta DESPUÉS del código síncrono — el Event Loop respeta ese orden por diseño.



Las Pestañas que Más se Usan



Elements

Inspeccionar y modificar el DOM y los estilos CSS en vivo



Console

Ejecutar JavaScript y ver errores/logs en tiempo real



Network

Ver cada petición HTTP realizada, su estado y contenido



Sources

Colocar breakpoints reales y depurar paso a paso

Hábito clave

Aprender a depurar con breakpoints en Sources ahorra muchísimo más tiempo a largo plazo que llenar el código de console.log.



Depurar sin un Solo console.log

```
app.js

function calcularTotal(precio, cantidad) {
  debugger;
  return precio * cantidad;
}
```



NOTA — La sentencia `debugger`; pausa la ejecución exactamente en ese punto cuando DevTools está abierto, permitiendo inspeccionar variables en ese instante.



Extensiones de Editor y DevTools Avanzado



GitLens

Etiqueta cada línea de código con su autor y fecha de cambio



Thunder Client

Simula peticiones HTTP (GET, POST) sin salir del editor



Network Throttling

Simula una conexión Slow 3G para probar estados de carga



Pestaña Application

Inspecciona y edita directamente el localStorage del navegador

Configuración de equipo

Guardar estas preferencias en `.vscode/settings.json` con `formatOnSave: true` estandariza el entorno de todo el equipo.



El Flujo Mínimo Diario

Terminal

```
git checkout -b feature/mi-cambio  
git add .  
git commit -m "Añade validacion de formulario"  
git push origin feature/mi-cambio
```



NOTA — Trabajar siempre en una rama feature/, nunca directamente en main — este hábito desde el primer día evita problemas graves más adelante.



Escribiendo Buenos Mensajes de Commit



Atómico

Un commit, un cambio lógico — no mezclar 3 cosas distintas en uno



Presente, no pasado

'Añade validación', no 'Añadido' ni 'Añadía'



Descriptivo pero breve

Que se entienda el 'qué' sin necesidad de abrir el diff



Evitar 'fix', 'cambios', 'wip'

Mensajes que no dicen nada útil al leerlos meses después

Prueba real

Si dentro de 6 meses alguien (o tú mismo) lee el mensaje sin ver el código, ¿entendería qué cambió y por qué?



GitFlow: Ramas, Conflictos y Pull Requests



main

Rama de producción estable; nunca se edita directamente



develop

Rama de integración donde confluyen las features terminadas



feature/...

Rama temporal para cada nueva funcionalidad o corrección



Pull Request

Petición para que otro compañero audite y apruebe el código antes de fusionar

Conflicto de fusión

Ocurre cuando dos personas editan la misma línea del mismo fichero; Git detiene la fusión y hay que resolverlo a mano antes de confirmar.



Así Trabajamos Aquí: VS Code + Docker



Nada se instala en local

Node, npm y Vite viven dentro del contenedor Docker, no en tu equipo



Dev Containers

VS Code se conecta directamente dentro del contenedor con la extensión Dev Containers



Terminal ya conectado

El terminal integrado de VS Code ES el terminal del contenedor — sin docker exec manual



Puerto reenviado

El puerto del contenedor (ej. 5173) se mapea solo a localhost en tu navegador

Cómo se siente

Trabajar así se siente exactamente igual que en local: abres la carpeta, abres un terminal, escribes npm run dev... la diferencia de que corre en Docker es invisible.



¿Qué es Vite? Más Allá de los Comandos



Qué es

Una herramienta de compilación ("build tool") y servidor de desarrollo para proyectos web modernos



Motor en desarrollo

Usa esbuild (escrito en Go) para un arranque casi instantáneo, sea cual sea el tamaño del proyecto



Motor en producción

Usa Rollup para generar el build final, optimizado y dividido en fragmentos pequeños



HMR

Hot Module Replacement: actualiza en el navegador solo el módulo que cambiaste, sin recargar la página ni perder el estado

Por qué importa

Antes de Vite, herramientas como Webpack recompilaban todo el proyecto en cada guardado — en proyectos grandes, eso eran varios segundos de espera por cada cambio.



Vite: Recarga Instantánea (en el Contenedor)

Terminal

```
npm create vite@latest mi-proyecto  
cd mi-proyecto && npm install  
npm run dev
```



NOTA — Este terminal ya está dentro del contenedor Docker. Vite arranca ahí, y VS Code reenvía el puerto a localhost:5173 en tu navegador automáticamente.



Herramientas que Automatizan Buenas Prácticas



ESLint

Detecta errores y patrones problemáticos antes de ejecutar el código



Prettier

Formatea el código automáticamente con un estilo consistente



Configuración compartida en equipo

Todo el equipo escribe con el mismo estilo sin discutirlo manualmente



Integración con el editor

Correcciones automáticas al guardar el fichero

Idea clave

Instalar ESLint y Prettier el primer día evita semanas de código inconsistente y discusiones estériles sobre estilo en el equipo.



Montar el Entorno de Trabajo Completo

1

Crear proyecto con Vite

Plantilla vanilla JS, en el terminal
del contenedor

2

Inicializar Git

Con .gitignore correcto
(node_modules incluido)

3

Instalar ESLint y Prettier

Configuración básica compartida

4

Provocar un error deliberado

Y depurarlo con un breakpoint, no
con console.log

5

Hacer el primer commit

Siguiendo las buenas prácticas
vistas hoy



ÉXITO — Criterio de éxito: el proyecto arranca con `npm run dev`, ESLint señala errores reales, y el primer commit sigue el formato correcto.



CASO REAL

El Día a Día de un Desarrollador Frontend Junior

- **Primera hora del día:** git pull, revisar notificaciones de PRs, planificar tareas
- **Depuración constante:** DevTools abierto la mayor parte de la jornada, no solo ante errores
- **Code review:** leer y comentar el código de compañeros es tan habitual como escribir el propio
- **Commits frecuentes y pequeños:** mejor que un único commit gigante al final del día
- **Aprendizaje continuo:** el ecosistema JavaScript evoluciona constantemente, la curiosidad es parte del oficio



Trampas al Empezar con JavaScript

Depurar solo con console.log

Mucho más lento que usar breakpoints reales en Sources

No entender el Event Loop

Provoca bugs de orden de ejecución difíciles de diagnosticar sin ese modelo mental

Ignorar los errores de la consola

Un error en rojo sin investigar suele ser la causa raíz de fallos posteriores

Copiar código sin entenderlo

Funciona hasta que hay que modificarlo o depurarlo, y entonces es un problema real



Trampas de Flujo de Trabajo

Trabajar directamente en main

Sin rama propia, cualquier error afecta inmediatamente al código principal

Commits gigantes al final del día

Imposibles de revisar y de revertir selectivamente si algo falla

node_modules versionado en Git

Infla el repositorio innecesariamente con dependencias regenerables

Sin ESLint/Prettier desde el inicio

El código se vuelve inconsistente progresivamente sin que nadie lo note a tiempo



Preguntas de Control y Debate

1 ¿Por qué `setTimeout(fn, 0)` no se ejecuta inmediatamente pese al tiempo 0?

2 ¿Qué ventaja tiene depurar con breakpoints frente a usar solo `console.log`?

3 ¿Por qué se recomienda trabajar siempre en una rama `feature/`, no en `main`?

4 ¿Qué problema resuelven ESLint y Prettier en un equipo de desarrollo?

5 ¿Por qué `node_modules` nunca debe versionarse en Git?



MINI-RETO PRÁCTICO

Reto Express (15-20 min, en parejas)

1

Crear un proyecto Vite

Desde cero, en el terminal del contenedor

2

Escribir una función con un bug

Intencionado, para depurar después

3

Depurar con breakpoint

Sin usar ni un solo console.log

4

Corregir y confirmar

El comportamiento esperado

5

Hacer 2 commits atómicos

Cada uno con su propio propósito claro



ÉXITO — Criterio de éxito: el bug se localiza y corrige usando exclusivamente breakpoints, y ambos commits siguen las buenas prácticas de mensaje.



Antes de Cerrar: El Entorno como Cimiento

- **Todo lo posterior depende de hoy:** un entorno mal configurado dificulta cada unidad siguiente
- **Los hábitos se fijan al principio:** es más fácil empezar bien que corregir malas costumbres después
- **DevTools se volverá algo natural:** con la práctica, dejará de sentirse como una herramienta externa



ÚLTIMA REFLEXIÓN

Una Pregunta Más Antes del Resumen

1

¿Qué diferencia notarías en tu forma de trabajar dentro de un mes si aplicas estos hábitos desde hoy?

Resumen de Puntos Clave

- ✓ JavaScript es de un solo hilo; el Event Loop gestiona la asincronía sin bloquear
- ✓ DevTools con breakpoints es más potente y rápido que depurar con console.log
- ✓ Git con ramas propias y commits atómicos es un hábito, no un extra opcional
- ✓ Vite acelera drásticamente el ciclo de desarrollo con recarga instantánea
- ✓ ESLint y Prettier deberían configurarse desde el primer commit del proyecto

➔ **SIGUIENTE** — UD2 — Sintaxis, Tipos y Funciones